

Laboratorio 8

Análisis de complejidad de algoritmos

Teoría de la computación

Introducción

Este documento contiene las respuestas del Laboratorio 8 que no son código: el análisis teórico de los problemas 1, 2 y 3 (parte a), los problemas 4 y 5, y los resultados de las mediciones de tiempo (parte b). El código con el que se hicieron las mediciones está en la carpeta `src` del repositorio.

Notación: $\lfloor x \rfloor$ es el piso de x , $\lceil x \rceil$ es el techo de x y $\log n$ es el logaritmo en base 2. En el lenguaje C, la división entre enteros (por ejemplo $n/2$) descarta la parte decimal, es decir, calcula $\lfloor n/2 \rfloor$.

Metodología de las mediciones

- Los programas están escritos en C y se compilaron con `gcc -O0` (sin optimización), para que el compilador no elimine los ciclos.
- El tiempo se mide con la función `clock()`. Si una ejecución dura menos de 0.05 s, se repite muchas veces y se calcula el promedio.
- En los problemas 2 y 3 se descarta la salida de `printf`, para que el tiempo no dependa de la velocidad de la pantalla.
- En el problema 1 la variable `counter` se declaró como `long long`, porque para n grande supera el límite de un `int`.
- Los valores de n que tardarían demasiado en medirse se estiman con la proporción de operaciones que predice el análisis teórico: $T(n) \approx T(n_{ref}) \cdot \frac{\text{operaciones}(n)}{\text{operaciones}(n_{ref})}$, donde n_{ref} es el mayor valor medido. Estos datos aparecen marcados como *estimado* en las tablas y con un círculo vacío en las gráficas.

Problema 1

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}
```

Parte a

Procedimiento. Se cuenta cuántas veces se ejecuta la instrucción más interna, `counter++`, analizando cada ciclo.

1. Ciclo de i : va desde $\lfloor n/2 \rfloor$ hasta n , por lo que se ejecuta $n - \lfloor n/2 \rfloor + 1 = \lceil n/2 \rceil + 1$ veces.
2. Ciclo de j : la condición $j + \lfloor n/2 \rfloor \leq n$ equivale a $j \leq n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$, por lo que se ejecuta $\lceil n/2 \rceil$ veces. Este valor no depende de i .
3. Ciclo de k : toma los valores $1, 2, 4, \dots, 2^m$ mientras $2^m \leq n$, es decir, mientras $m \leq \log n$. Se ejecuta $\lfloor \log n \rfloor + 1$ veces. Este valor no depende de i ni de j .

Como los límites de cada ciclo no dependen de los otros, el total de ejecuciones es el producto de las tres cantidades:

$$T(n) = (\lceil n/2 \rceil + 1) \cdot \lceil n/2 \rceil \cdot (\lfloor \log n \rfloor + 1).$$

Para $n \geq 2$ se pueden acotar los factores:

- Cota superior: $\lceil n/2 \rceil + 1 \leq n$, $\lceil n/2 \rceil \leq n$ y $\lfloor \log n \rfloor + 1 \leq 2 \log n$. Entonces $T(n) \leq 2n^2 \log n$.
- Cota inferior: $\lceil n/2 \rceil + 1 \geq n/2$, $\lceil n/2 \rceil \geq n/2$ y $\lfloor \log n \rfloor + 1 \geq \log n$. Entonces $T(n) \geq \frac{1}{4}n^2 \log n$.

Por lo tanto, $\frac{1}{4}n^2 \log n \leq T(n) \leq 2n^2 \log n$ para $n \geq 2$, y $T(n) = \Theta(n^2 \log n)$.

Comprobación. Para $n = 10$: $(5 + 1) \cdot 5 \cdot (3 + 1) = 120$. Para $n = 100$: $51 \cdot 50 \cdot 7 = 17850$. Ambos valores coinciden con el contador que devuelve el programa (ver Tabla 1).

Respuesta: la complejidad de tiempo es $O(n^2 \log n)$.

Parte b

La Tabla 1 muestra el tamaño de input contra el tiempo de ejecución. La columna *Operaciones* es el valor exacto de `counter` (el programa lo devuelve y coincide con la fórmula $T(n)$ de la parte a). La Figura 1 muestra los mismos datos.

Tabla 1: Problema 1: tamaño de input vs tiempo.

n	Operaciones	Tiempo (s)	Tipo de dato
1	2	8.00×10^{-9}	medido
10	120	2.60×10^{-7}	medido
100	17 850	4.51×10^{-5}	medido
1 000	2 505 000	6.68×10^{-3}	medido
10 000	350 070 000	9.36×10^{-1}	medido
100 000	42 500 850 000	1.14×10^2	medido
1 000 000	5 000 010 000 000	1.34×10^4	estimado

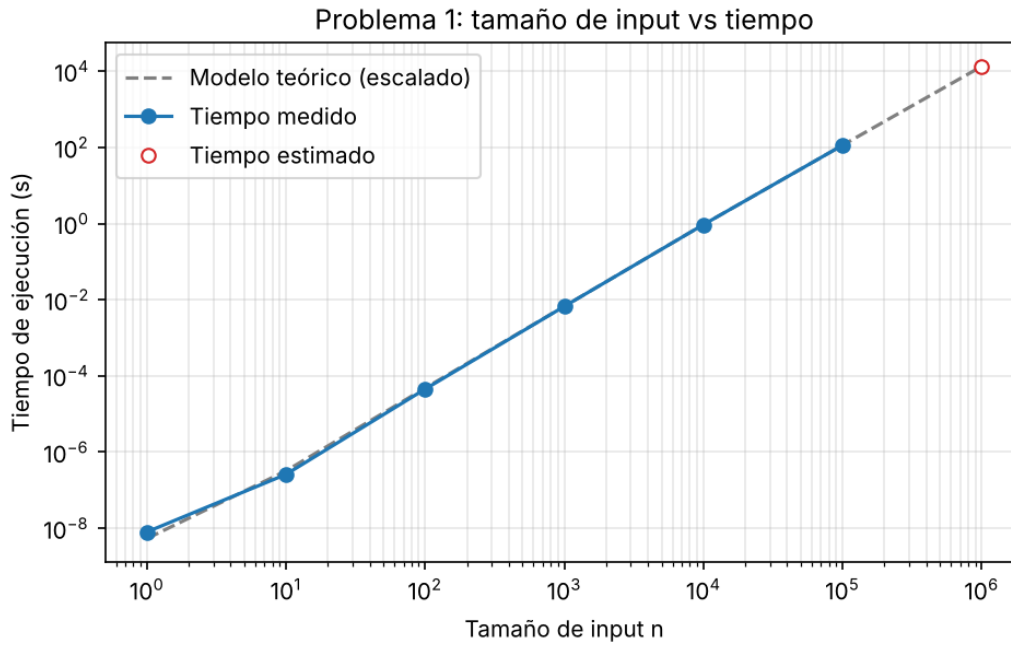


Figura 1: Problema 1: tamaño de input vs tiempo (ambos ejes en escala logarítmica).

Análisis. Cada vez que n se multiplica por 10, el número de operaciones se multiplica por un factor un poco mayor que 100, debido al factor $\log n$: 139.7 entre $n = 1\,000$ y $n = 10\,000$, y 121.4 entre $n = 10\,000$ y $n = 100\,000$. El tiempo medido crece casi en la misma proporción (140.1 y 121.6, respectivamente), lo que concuerda con $O(n^2 \log n)$.

El valor para $n = 1\,000\,000$ **no se midió**: ejecutar el programa tomaría aproximadamente 3.7 horas, por lo que se estimó con la proporción de operaciones (ver la metodología). Para $n = 1$ y $n = 10$ los tiempos son de nanosegundos y están dominados por el costo de llamar a la función y de medir.

Problema 2

```
void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            break;
        }
    }
}
```

Parte a

Procedimiento. Se analizan los dos casos posibles.

1. Si $n \leq 1$, la función termina de inmediato. Tarda un tiempo constante.
2. Si $n \geq 2$, el ciclo externo se ejecuta n veces. En cada una de ellas el ciclo interno empieza con $j = 1$, cumple la condición $1 \leq n$, ejecuta `printf` y la instrucción `break` lo termina. Por lo tanto, el ciclo interno hace **exactamente una iteración** y no n .

Sea c_2 el costo constante de una vuelta del ciclo externo (inicializar j , comparar, imprimir y salir con `break`) y c_1 el costo constante de la preparación de la función. Entonces, para $n \geq 2$:

$$T(n) = c_1 + c_2 \cdot n = \Theta(n).$$

La función imprime `Sequence` exactamente n veces cuando $n \geq 2$.

Respuesta: la complejidad de tiempo es $O(n)$.

Parte b

La Tabla 2 muestra el tamaño de input contra el tiempo de ejecución. La columna *Operaciones* es el número de veces que se ejecuta `printf`. La Figura 2 muestra los mismos datos. En este problema todos los valores de n se midieron.

Tabla 2: Problema 2: tamaño de input vs tiempo.

n	Operaciones	Tiempo (s)	Tipo de dato
1	0	2.00×10^{-9}	medido
10	10	1.20×10^{-7}	medido
100	100	1.22×10^{-6}	medido
1 000	1 000	1.14×10^{-5}	medido
10 000	10 000	1.13×10^{-4}	medido
100 000	100 000	1.15×10^{-3}	medido
1 000 000	1 000 000	1.16×10^{-2}	medido

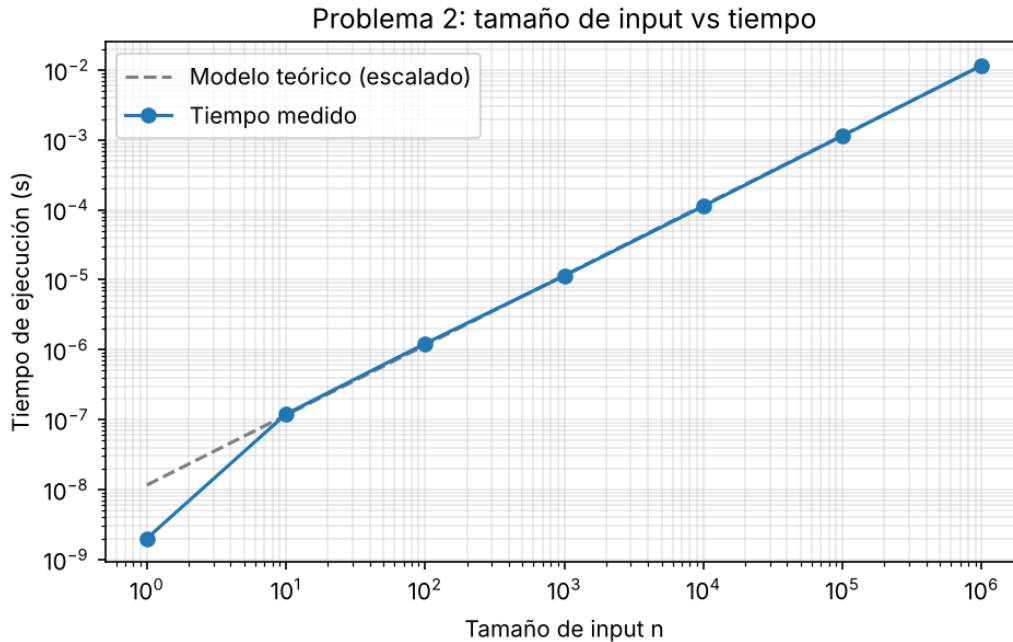


Figura 2: Problema 2: tamaño de input vs tiempo (ambos ejes en escala logarítmica).

Análisis. El tiempo crece de forma lineal: cada vez que n se multiplica por 10, el tiempo se multiplica por un factor cercano a 10 (por ejemplo, de 1.15×10^{-3} s a 1.16×10^{-2} s entre $n = 100\,000$ y $n = 1\,000\,000$). Esto concuerda con $O(n)$. Para $n = 1$ la función regresa de inmediato y el tiempo es de apenas unos nanosegundos.

Problema 3

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
        }
    }
}
```

Parte a

Procedimiento. Se cuenta cuántas veces se ejecuta `printf`.

1. Ciclo de i : va desde 1 hasta $\lfloor n/3 \rfloor$, por lo que se ejecuta $\lfloor n/3 \rfloor$ veces.
2. Ciclo de j : toma los valores $1, 5, 9, \dots, 1 + 4t$ mientras $1 + 4t \leq n$, es decir, $t \leq (n-1)/4$. Se ejecuta $\lfloor (n-1)/4 \rfloor + 1 = \lceil n/4 \rceil$ veces. Este valor no depende de i .

El total de ejecuciones es el producto:

$$T(n) = \lfloor n/3 \rfloor \cdot \lceil n/4 \rceil.$$

Para $n \geq 6$ se cumple $\lfloor n/3 \rfloor \geq n/3 - 1 \geq n/6$ y $\lceil n/4 \rceil \geq n/4$, luego $T(n) \geq n^2/24$. Además, $\lfloor n/3 \rfloor \leq n/3$ y $\lceil n/4 \rceil \leq n/4 + 1 \leq n/2$ (para $n \geq 4$), luego $T(n) \leq n^2/6$. Así:

$$\frac{n^2}{24} \leq T(n) \leq \frac{n^2}{6} \quad (n \geq 6) \quad \implies \quad T(n) = \Theta(n^2).$$

Comprobación. Para $n = 10$: $3 \cdot 3 = 9$ impresiones. Para $n = 100$: $33 \cdot 25 = 825$ impresiones.

Respuesta: la complejidad de tiempo es $O(n^2)$.

Parte b

La Tabla 3 muestra el tamaño de input contra el tiempo de ejecución. La columna *Operaciones* es el número de veces que se ejecuta `printf`, que coincide con $T(n) = \lfloor n/3 \rfloor \cdot \lceil n/4 \rceil$. La Figura 3 muestra los mismos datos. En este problema todos los valores de n se midieron (el último tardó unos 16 minutos).

Tabla 3: Problema 3: tamaño de input vs tiempo.

n	Operaciones	Tiempo (s)	Tipo de dato
1	0	2.00×10^{-9}	medido
10	9	1.09×10^{-7}	medido
100	825	1.02×10^{-5}	medido
1 000	83 250	9.67×10^{-4}	medido
10 000	8 332 500	1.18×10^{-1}	medido
100 000	833 325 000	9.77×10^0	medido
1 000 000	83 333 250 000	9.62×10^2	medido

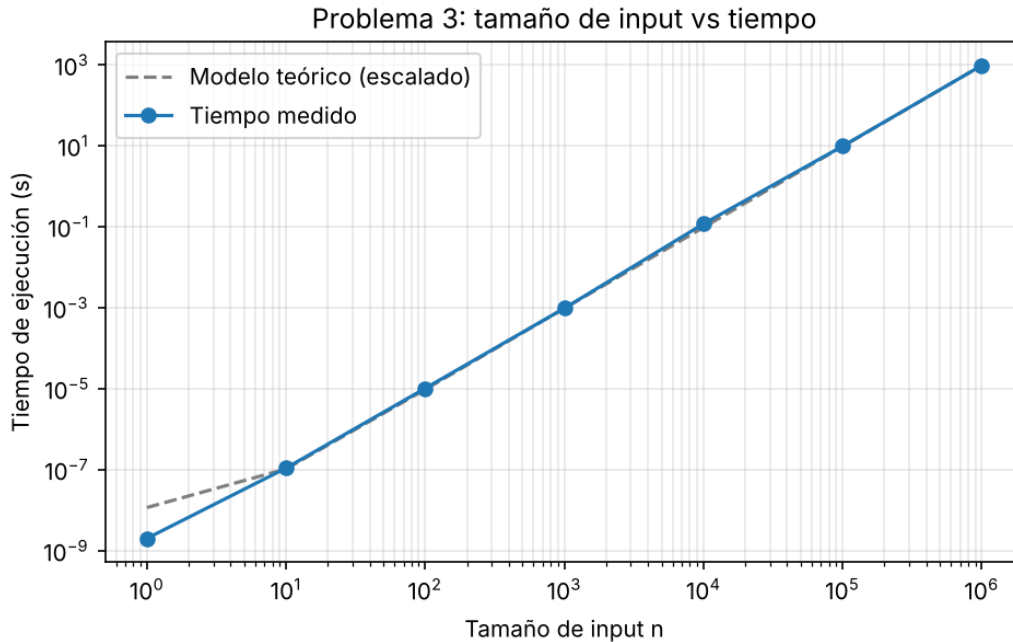


Figura 3: Problema 3: tamaño de input vs tiempo (ambos ejes en escala logarítmica).

Análisis. Cada vez que n se multiplica por 10, el tiempo se multiplica por un factor cercano a 100, como corresponde a un crecimiento cuadrático. Entre $n = 100\,000$ y $n = 1\,000\,000$ el factor medido fue 98.5. En los tramos anteriores el factor varió entre 83 y 122 por las variaciones propias de la medición, pero en conjunto, de $n = 1\,000$ a $n = 1\,000\,000$, el tiempo creció unas 995 000 veces y las operaciones unas 1 001 000 veces. Esto concuerda con $O(n^2)$.

Validación de la estimación. Antes de medir $n = 1\,000\,000$, el método de estimación de la metodología predecía 976.8 s a partir del valor medido para $n = 100\,000$. El tiempo medido fue 962.3 s, una diferencia de aproximadamente 1.5 %. Esto respalda el uso de la estimación para el valor que no se pudo medir en el problema 1.

Problema 4

Se analizan los tres algoritmos mencionados en el enunciado. En todos los casos n es el número de elementos del arreglo y se cuentan las comparaciones entre elementos.

Búsqueda lineal (Linear Search)

El algoritmo recorre el arreglo desde el primer elemento y se detiene cuando encuentra el valor buscado x .

- **Mejor caso:** x está en la primera posición. Se hace 1 comparación, $T(n) = 1 = \Theta(1)$.
- **Peor caso:** x está en la última posición o no está en el arreglo. Se hacen n comparaciones, $T(n) = n = \Theta(n)$.

- **Caso promedio:** se supone que x está en el arreglo y que cualquier posición tiene la misma probabilidad, $1/n$. Si está en la posición i se hacen i comparaciones, así que

$$T_{prom}(n) = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n).$$

Si la búsqueda puede fallar, el promedio queda entre $(n+1)/2$ y n , que también es $\Theta(n)$.

Búsqueda binaria (Binary Search)

El arreglo debe estar ordenado. En cada paso se compara x con el elemento central y se descarta la mitad del arreglo.

- **Mejor caso:** x es el elemento central. Se hace 1 comparación, $T(n) = \Theta(1)$.
- **Peor caso:** se llega hasta un subarreglo de un solo elemento. Cada comparación reduce el tamaño a la mitad, así que $T(n) = T(\lfloor n/2 \rfloor) + 1$ con $T(1) = 1$. Al repetir la reducción se obtiene $T(n) = \lfloor \log n \rfloor + 1 = \Theta(\log n)$.
- **Caso promedio:** se toma $n = 2^k - 1$ y se supone que cada posición es igual de probable. El árbol de decisiones tiene en el nivel l exactamente 2^{l-1} elementos, y cada uno se encuentra con l comparaciones. El total es

$$\sum_{l=1}^k l \cdot 2^{l-1} = (k-1)2^k + 1$$

(se comprueba por inducción: para $k = 1, 2, 3$ da 1, 5, 17). Entonces

$$T_{prom}(n) = \frac{(k-1)2^k + 1}{2^k - 1} = (k-1) + \frac{k}{2^k - 1} \approx \log(n+1) - 1 = \Theta(\log n).$$

Quick sort

El algoritmo escoge un pivote y reparte los demás elementos en dos grupos (menores y mayores). Repartirlos cuesta $n - 1$ comparaciones. Luego ordena cada grupo de forma recursiva.

- **Mejor caso:** el pivote siempre queda en el centro, y los grupos tienen tamaño cercano a $n/2$. Entonces $T(n) \approx 2T(n/2) + n$. Hay $\log n$ niveles de recursión y en cada nivel se hacen cerca de n comparaciones, así que $T(n) = \Theta(n \log n)$ (también se obtiene con el teorema maestro).
- **Peor caso:** el pivote siempre es el menor o el mayor elemento (por ejemplo, un arreglo ya ordenado si el pivote es el primer elemento). Un grupo queda vacío y el otro tiene $n - 1$ elementos: $T(n) = T(n-1) + (n-1)$ con $T(1) = 0$. Entonces

$$T(n) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2).$$

- **Caso promedio:** se supone que todas las permutaciones son igual de probables, por lo que el pivote termina en cada posición con probabilidad $1/n$:

$$T(n) = (n-1) + \frac{1}{n} \sum_{k=0}^{n-1} [T(k) + T(n-1-k)] = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} T(k).$$

Se multiplica por n : $nT(n) = n(n-1) + 2\sum_{k=0}^{n-1} T(k)$. Se resta la misma ecuación para $n-1$:

$$nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1) \implies nT(n) = (n+1)T(n-1) + 2(n-1).$$

Se divide entre $n(n+1)$ y se repite la ecuación hasta $T(0) = 0$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)} \implies \frac{T(n)}{n+1} = \sum_{k=1}^n \frac{2(k-1)}{k(k+1)} = \sum_{k=1}^n \left(\frac{4}{k+1} - \frac{2}{k} \right).$$

Con el número armónico $H_n = \sum_{k=1}^n 1/k$ la suma vale $2H_n + \frac{4}{n+1} - 4$, de donde

$$T(n) = 2(n+1)H_n - 4n \approx 2n \ln n \approx 1.39n \log n = \Theta(n \log n).$$

(Comprobación: $T(2) = 2 \cdot 3 \cdot 1.5 - 8 = 1$ comparación, que es correcto.)

Resumen del problema 4.

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Búsqueda binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Quick sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

Problema 5

En los tres incisos se supone que las funciones son no negativas para n suficientemente grande.

Inciso a) Verdadero

Enunciado: si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.

Justificación. Por definición existen constantes positivas c_1, c_2, d_1, d_2 y un valor n_0 tales que, para todo $n \geq n_0$:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{y} \quad d_1 h(n) \leq g(n) \leq d_2 h(n).$$

Combinando las desigualdades:

$$f(n) \leq c_2 g(n) \leq c_2 d_2 h(n) \quad \text{y} \quad f(n) \geq c_1 g(n) \geq c_1 d_1 h(n).$$

Despejando $h(n)$:

$$\frac{1}{c_2 d_2} f(n) \leq h(n) \leq \frac{1}{c_1 d_1} f(n) \quad (n \geq n_0),$$

que es la definición de $h(n) = \Theta(f(n))$. En otras palabras, Θ es una relación simétrica y transitiva.

Inciso b) Verdadero

Enunciado: si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.

Justificación. Existen constantes positivas c y d y un valor n_0 tales que, para todo $n \geq n_0$, $f(n) \leq c g(n)$ y $g(n) \leq d h(n)$. Entonces

$$f(n) \leq c g(n) \leq c d h(n) \implies h(n) \geq \frac{1}{cd} f(n) \quad (n \geq n_0),$$

que es la definición de $h(n) = \Omega(f(n))$.

Inciso c) Falso

Enunciado: $f(n) = \Theta(n^2)$, donde $f(n)$ es el tiempo de ejecución del programa:

```
def A(n):  
    atupla = tuple(range(0, n))  
    S = set()  
    for i in range(0, n):  
        for j in range(i + 1, n):  
            S.add(atupla[i:j])
```

Justificación. Los dos ciclos se ejecutan $\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$ veces, lo que haría pensar en $\Theta(n^2)$. Sin embargo, cada iteración no cuesta un tiempo constante:

- `atupla[i:j]` crea una tupla nueva y copia sus $j-i$ elementos, así que cuesta $\Theta(j-i)$.
- `S.add(...)` necesita calcular el hash de esa tupla, que se obtiene a partir de todos sus elementos, así que también cuesta $\Theta(j-i)$.
- Todas las tuplas son distintas (cada una tiene un inicio y un largo diferentes), por lo que ninguna inserción se evita.

Entonces el costo de cada iteración es $\Theta(j-i)$. Si $d = j-i$, hay $n-d$ pares (i, j) con esa diferencia, y el tiempo total es

$$f(n) = \Theta\left(\sum_{d=1}^{n-1} d(n-d)\right) = \Theta\left(\frac{n(n-1)(n+1)}{6}\right) = \Theta(n^3).$$

La creación inicial de `atupla` cuesta $\Theta(n)$ y no cambia el resultado. Como n^3 crece más rápido que n^2 , $f(n) \neq \Theta(n^2)$. El enunciado sería cierto solamente si cortar y calcular el hash de una tupla costara $O(1)$, y no es así.